ChatGPT

Ideal Tech Stack & Scalable Architecture for Algorithm Battle Arena

Algorithm Battle Arena is a real-time competitive coding platform (students vs. teachers) requiring live match lobbies, chat, code execution sandboxing, leaderboards, and rich user roles. To launch as a **low-cost startup** and scale if successful, the stack should leverage free tiers and open-source components, support heavy WebSocket usage, and allow easy CI/CD and expansion (AI, analytics, etc.).

Recommended Architecture Overview

The core architecture uses a **microservice-friendly Node.js backend** with WebSockets and Docker sandboxes, a modern **React/Next.js frontend**, and cloud-hosted services (DB, auth, hosting). A load balancer fronts multiple Node.js/WebSocket servers. These servers share state via Redis or a message broker and a common database [1]. Code submissions are executed in isolated containers (e.g. a Judge0 service) to ensure security and scalability [2] [3]. Users (students, teachers, admins) authenticate via a managed service (e.g. Supabase or Auth0). The diagram below illustrates a high-level deployment:

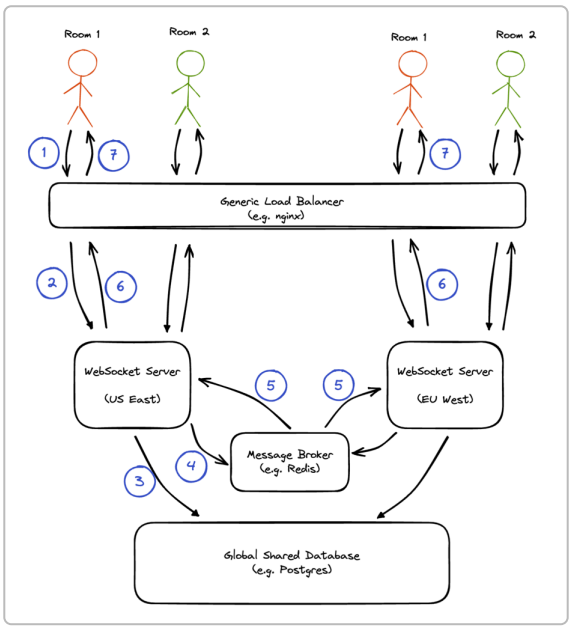

Figure: Scalable real-time architecture (WebSocket servers behind a load balancer, sharing state via Redis and a central DB) [1].

In this design:

- **Frontend:** A **React** (TypeScript) app—ideally using **Next.js** for SEO/SSR—runs on Vercel/Netlify. React is component-driven and highly performant, with seamless deployment on Vercel's generous free

tier [4]. Next.js API routes or separate serverless functions handle light backend logic (e.g. fetching problems, leaderboards) on demand. This decoupling enables continuous deployment: a push to the repo auto-deploys new versions [5].

- **Backend API:** The main API layer is **Node.js** (with TypeScript) using Express or a framework like NestJS. Node's non-blocking event loop excels at handling concurrent I/O (thousands of sockets) [6], making it ideal for real-time chat/match updates. We design it as microservices (e.g. auth service, lobby/match service, user service, grading service), ensuring **scalability and fault tolerance** [7] [8]. For example, CodeSpace's architecture is *"microservice-ready"* for scaling [7]. Each service can be deployed in Docker containers (e.g. on AWS ECS/Fargate or Google Cloud Run), or as clustered processes, allowing independent scaling (e.g. spin up more grading servers when load increases).

- **Real-Time (WebSockets):** All multiplayer features use WebSockets (e.g. via **Socket.IO**) for bidirectional updates. WebSockets provide persistent, low-latency channels (unlike stateless HTTP) [9], which is essential for live code duels and chat. Socket.IO (Node) simplifies rooms and event broadcasting. In production, we run multiple WebSocket server instances behind a load balancer (with sticky sessions or a shared session store). A Redis (or RabbitMQ) message broker routes events between servers so that users in the same game ("rooms") stay in sync [1]. This pattern (load balancer + message broker + shared DB) is a common, scalable approach for WebSockets [1].

- **Database:** Use a hosted database with a **free-tier** (e.g. **Supabase/PostgreSQL** or **MongoDB Atlas**). PostgreSQL is recommended for relational features (RBAC, transactional safety for score updates, complex queries), and Supabase provides it plus built-in auth. Supabase's free plan includes 500 MB DB and 50k MAU [10], which should suffice early on. (MongoDB Atlas has a similar free tier if a NoSQL model is preferred.) All game state (users, problems, match history, leaderboards) lives here. Caching layers (Redis or in-memory) can speed up hot reads (e.g. leaderboard queries) and store ephemeral state.

- **Authentication & User Management:** Leverage a managed auth system to reduce development overhead. Options include **Supabase Auth**, **Clerk**, or **Auth0**, each offering free tiers. These handle email/password (or SSO), password reset, role claims, etc. For example, Alpha4Coders' CodeBattle uses Clerk for secure auth [11]. Roles (student/teacher/admin) can be represented as custom claims or in the user table. This enables teacher dashboards and admin controls without building a user system from scratch.

- **Code Execution Engine:** A critical service is the **sandboxed code runner**. Use an **open-source engine** like **Judge0** or **Piston**. Each code submission is sent to a dedicated sandbox container (e.g. a Docker container with limited CPU/memory) to compile/run tests [2] [3]. Judge0 is explicitly designed for this use case: *"the open source code execution engine powering millions of code executions worldwide. Robust, scalable, and secure."* [3]. It can be self-hosted (on AWS/GCP) to avoid per-execution costs, or one can start with Judge0's free hosted API for rapid MVP. Using Docker isolation (as shown in industry examples) ensures each user's code cannot harm the host [2] [12].

- **Infrastructure & Hosting:** Host the **frontend** on Vercel or Netlify (both have free static/SSR hosting). Host the **backend/API** on a cloud provider: options include AWS (EC2, ECS, Fargate, or App Runner), Google Cloud (Cloud Run), or Render. Early on, a single virtual server (or small container cluster) can suffice. Use Docker for all services to enable easy deployment. Services like Redis (for socket

adapter/pub-sub) and PostgreSQL can use managed offerings (AWS ElastiCache, RDS) or hosted PaaS like Supabase. Start with free-tier or low-cost instances, and design so additional instances can be launched behind a load balancer as needed.

- **CI/CD and Testing:** Use **GitHub Actions** (free for open-source) to automate testing and deployment. On each pull request, run unit tests (Jest for backend, Cypress/Playwright for UI) and lint checks. On merge to `main`, auto-deploy: e.g. push the frontend to Vercel and the backend Docker image to a registry then ECS/Cloud Run. CodeSpace's project uses CI/CD via Vercel/Azure [5]; our pipeline can be similarly automated.

- **Payment & Analytics:** Integrate **Stripe** for subscription plans. Stripe's SDK and Payment Links allow quick setup of paid tiers (e.g. premium matchmaking, advanced analytics). Stripe charges no monthly fee (only processing fees) and has generous developer tooling [13]. For analytics, use a combination of built-in logging and a BI tool (e.g. Metabase or Google Analytics) to track user engagement and retention. Expose dashboards for admins/teachers via a protected UI.

Architecture Diagram

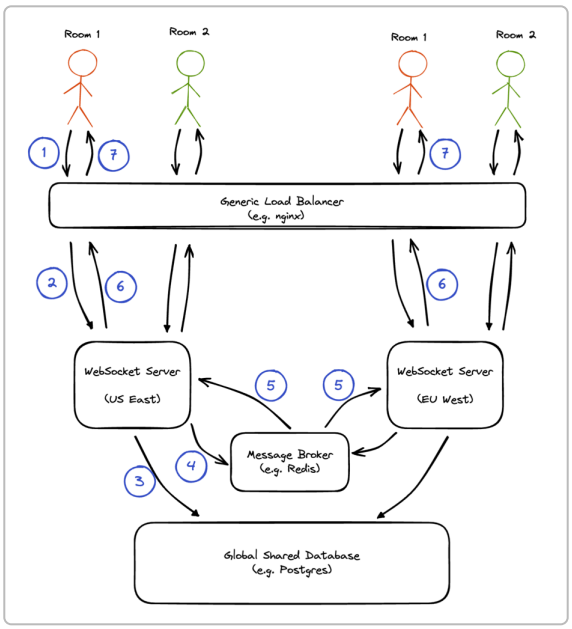

Figure: High-level system architecture. A load balancer directs traffic to multiple Node.js servers. WebSocket servers coordinate via Redis (pub/sub) and a shared Postgres database. A separate Code Execution microservice (e.g. Judge0) handles code runs in Docker containers. Other services include Auth, Payments (Stripe), and Analytics.

In practice, each Node.js instance handles HTTP API calls and also attaches to Socket.IO for real-time events. Redis (or a similar message broker) is used as a Socket.IO adapter and cache to synchronize events across instances. The **global database** (e.g. Postgres) stores persistent data (users, problems, scores). The code runner service pulls new submissions from a job queue, executes them, and returns results via the API. This decoupled design (microservices + containers) gives **flexibility and fault isolation** [8] [2].

Core Components & Justification

- **React/Next.js Frontend:** Modern, component-based UI. Next.js offers both static site generation and API routes, simplifying deployment. React's ecosystem and tooling (TypeScript support, rich libraries) boost developer productivity. This matches patterns seen in competitor projects (e.g. CodeSpace uses React for a *"responsive and component-driven UI built for speed"* [4]). Vercel's free hosting and preview URLs streamline QA and releases.

- **Node.js Backend (Express/Nest) + Socket.IO:** Node's single-threaded, event-driven model is **highly effective for real-time apps** [6]. It handles many concurrent connections with low latency (Netflix, Uber and others use it for this reason [14]). Socket.IO on Node makes implementing chat/matchrooms straightforward. We can structure the backend with REST (or GraphQL) APIs for non-real-time data and a separate WebSocket service for live events. This is analogous to Alpha4Coders' CodeBattle (Node+Express, Socket.IO) [11].

- **Microservices & Docker:** We adopt a microservice architecture to improve scalability and maintainability [7] [8]. For example, separate services can handle **matchmaking**, **match lifecycle**, **grading/submission**, **profile/analytics**, etc. Each service runs in its own Docker container, isolating failures and improving security. As one medium analysis notes, isolating services in containers *"minimizes the risks associated with executing unsafe code"* [2]. We can start with a monolith for MVP speed and split into microservices as load grows.

- **Database (PostgreSQL or MongoDB):** We recommend a managed **PostgreSQL** for relational data integrity (schedules, permissions). Supabase offers a free Postgres instance (500 MB, 50k users) plus realtime listeners. Alternatively, **MongoDB Atlas** (free tier) is a popular choice if a flexible schema is desired (as used by CodeBattle's example [11]). The chosen DB should support indexing on fields like ELO rating, timestamps, etc., and handle moderate write load (score updates, new problems). Use Redis or Memcached as a cache layer for hot reads (leaderboard lookups, UI state).

- **Authentication:** Use Supabase Auth, Clerk, or Auth0 to manage users, passwords, and roles. Supabase Auth tightly integrates with the DB and has a generous free tier. Clerk (used by CodeBattle) and Auth0 also simplify multi-role login flows. This avoids building authentication from scratch. We store roles (student/teacher/admin) in user metadata or a dedicated table to enforce permissions.

- **Sandboxed Code Execution:** Each code submission is queued and processed by a **code-runner service**. This service spins up a Docker container per submission (with CPU/memory limits), executes the code, collects results, and destroys the container. Such an isolated approach is proven: platforms using Docker for each run "improves security, stability and scalability" [12] [2]. An open-source engine like **Judge0** handles language support (40+ langs) and result streaming. The Judge0 team notes their engine is *"robust, scalable, and secure"* [3]. For an MVP, one can use a hosted Judge0 or Piston API (as CodeBattle did [11]) and later switch to self-hosting.

- **Real-time Communication:** As noted, WebSockets (via Socket.IO) handle live multiplayer interactions. We maintain rooms (one per battle lobby), broadcast code updates and chat messages instantly, and update leaderboards. To scale multiple WebSocket servers, we use Redis Pub/Sub so events on one instance reach all necessary clients [1]. This follows best practices: e.g. one design uses a load balancer, Nginx, and Redis to distribute WebSocket traffic and state [1].

- **DevOps / CI/CD:** We use GitHub Actions for continuous integration. On each push, run linting/tests and build Docker images. On merges to main, automatically deploy: e.g. `git push` triggers Vercel (frontend) and updates backend services (via container registry to cloud). Vercel and cloud providers integrate with GitHub for seamless preview deployments. Testing frameworks include Jest for backend and Cypress/Playwright for end-to-end UI tests.

- **Caching & Scaling:** Use Redis not only for sockets but also caching (e.g. session store, rate-limiting, problem metadata). If traffic spikes, we can horizontally scale stateless services behind a load balancer. Stateless scaling is simple for HTTP; for WebSockets we require sticky sessions or an elastic message broker. In practice, services like Kubernetes or AWS ECS handle this. Start small (one t2.medium or equivalent) and add instances as users grow.

- **Monitoring & Analytics:** Integrate logging (e.g. AWS CloudWatch or GCP Stackdriver) for backend errors. Use a metrics dashboard (Prometheus + Grafana, or a SaaS like Datadog) to track system health. For product analytics, use Google Analytics or Mixpanel to see feature usage, and custom analytics tables (or Metabase) for admin reporting (e.g. student progress, popular problems).

Alternative Stacks (with Trade-offs)

While the above is our **primary** recommendation, here are some alternate approaches:

- **Serverless/JAMstack (Next.js + Supabase/Vercel):** For a lean MVP, build the front-end in Next.js and use Next API Routes or serverless functions for the backend. Use **Supabase** (Auth + Postgres) for DB and user management [10]. This requires *no self-hosted servers*: Vercel's free tier hosts the app, and Supabase hosts the DB/auth. **Pros:** Very low maintenance, instant scalability of front-end, free tier is generous [10]. **Cons:** WebSockets are not natively supported by static hosts (one would need an external WebSocket service like Pusher or a separate WebSocket server), and serverless time limits can complicate long-running grading jobs. This stack also ties you to the Supabase/Vercel ecosystem. (This approach is akin to Joe Osborne's free SaaS stack [10].)

- **Monolithic Node.js (MERN):** Use a single Node.js/Express server with MongoDB and Socket.IO. **Pros:** Simplest to start, all-in-one codebase. Node's event model handles real-time well [6]. MongoDB Atlas has a free tier as well. **Cons:** As traffic grows, a monolith can become a bottleneck. Scaling requires deploying more copies (with sticky sessions or a message queue). Managing sessions and state in one service is harder. Also, reliability depends on one service. This is easier initially but sacrifices the scalability benefits of microservices.

- **Python/Django + Channels:** Use Django for the backend (with Django REST Framework for APIs and **Channels** for WebSockets). **Pros:** Django's admin and ORM accelerate building dashboards and managing content. Rich ecosystem (beautiful admin UI, many plugins). **Cons:** Python's async support is newer; scaling Channels requires Redis (for Django's channel layer) anyway. For a high-concurrency real-time game, Node has an edge: in head-to-head comparisons "for use cases demanding high concurrency, Node.js holds a significant edge" [15]. Also, fewer real-time tutorials exist for Python than for Node, so development may be slower.

- **Elixir/Phoenix Framework:** For extreme concurrency, Elixir's lightweight processes and built-in channels are ideal (Phoenix was built for chat/gaming). **Pros:** Incredible support for millions of websockets, fault-tolerance, hot upgrades. **Cons:** Smaller talent pool; new platform might slow hiring. Libraries for code execution (sandboxing) would still need integration (e.g. call out to a Docker runner). Not mainstream, but worth a mention for maximum scalability.

- **Firebase/Firestore + Cloud Functions:** Google's Firebase can be used to host the entire stack: Firestore DB, Firebase Auth, Cloud Functions (NodeJS) as the backend logic, and Firebase Realtime Database or Firestore real-time subscriptions for push. **Pros:** Built-in real-time sync, easy auth, generous free tier. **Cons:** Running arbitrary code is non-trivial (would require Cloud Run or Cloud Functions with security risk). Lock-in to Google's platform, and cost can grow unpredictably with usage. Overall, great for a quick prototyping but potentially limiting for a coding platform.

Each alternative has trade-offs in cost, ease-of-use, and scalability. For example, Next.js+Supabase is extremely quick to launch for free [10], but might need bolt-ons for real-time features. Python/Django offers rapid development with batteries included, but requires extra work to match Node's real-time performance [15]. We consider the Node.js/microservices approach the *most balanced* for our needs.

Future Growth Features

As the product matures, the following enhancements can fuel growth and monetization:

- **AI Tutoring & Code Assistance:** Integrate GPT or similar LLMs to give contextual hints, analyze code, and personalize learning. For example, CodeSpace envisions *"AI-assisted problem recommendations"* and *"code analysis and feedback using AI models"* [16]. Alpha4Coders' CodeBattle employs Google Gemini to provide real-time code hints and debugging suggestions [17]. Our platform could offer a chatbot tutor or inline annotations (e.g. "lightbulb hints") powered by GPT. This would greatly enhance learning for beginners.

- **Gamification:** Add badges, achievements, seasons/tournaments and ELO-based rankings. CodeSpace plans *"achievement badges and streak rewards"* and seasonal leaderboards [18]. We can introduce levels (novice→grandmaster), daily/weekly challenges, and rewards for consistency. This drives engagement and retention. Virtual "tournaments" (1v1 or all-users contests) can be staged with prizes to build community.

- **Mobile App / PWA:** Build a mobile-friendly interface or standalone app (React Native or Flutter) so users can practice on the go. Push notifications (for match invites or streak reminders) would increase engagement.

- **Advanced Analytics & Admin Dashboards:** Expand reporting for teachers and admins. Provide classroom-level stats (students' strengths/weaknesses), retention analytics, and usage trends. Tools like Metabase or a custom dashboard can show teacher how their students progress. Administrators can get revenue reports, system health metrics, and A/B test results to drive business decisions.

- **Internationalization & Accessibility:** Support multiple languages (UI translations and even localized problems) to grow internationally. Ensure the platform meets accessibility standards (ARIA roles, keyboard nav) so all students can use it.

- **Content Ecosystem & Community:** Integrate blogs/forum (as CodeSpace did with a Medium-like blog system [19]) so users share knowledge. Allow users to author and submit problems or tutorials. This builds a community and content library, making the platform sticky.

- **Analytics for Engagement:** Use machine learning to recommend problems or pair opponents. For example, match opponents of similar skill (as envisaged by CodeSpace) [16], or suggest next topics based on a student's performance trend.

By building on this flexible stack, Algorithm Battle Arena can evolve rapidly. Early MVP functionality focuses on robust real-time play and basic analytics. As traction grows, adding AI-driven tutoring, deeper gamification, mobile support, and rich analytics will drive user acquisition and justify premium subscriptions.

Sources: We base these choices on modern SaaS best practices and similar platforms. For instance, CodeSpace's architecture highlights Node.js with WebSockets and Docker sandboxes [7], and industry advice emphasizes microservices for scalability [8]. The benefits of WebSockets (persistent, low-latency connections for chat/gaming) are well-documented [9] [20]. And open-source tools like Judge0 provide proven code execution sandboxing [3]. These references and others inform the above architecture.

---

[1] Scalable WebSocket Architecture Architecture Diagram - Software Diagrams
https://softwarediagrams.com/diagrams/scalable-websocket-architecture/

[2] [8] [12] Scalable Platform for Remote Code Execution: Architecture, Technologies, and Implementation | by Oleksii Bondar | Medium
https://medium.com/@oleksiijko/scalable-platform-for-remote-code-execution-architecture-technologies-and-implementation-4c0e8da72a8e

[3] Introducing the Judge0 blog | Code execution, tutorials & updates
https://judge0.com/blog/introducing-the-judge0-blog

[4] [5] [7] [16] [18] [19] GitHub - sambhandavale/code_space: Transforming coding challenges into engaging competitions.
https://github.com/sambhandavale/code_space

[6] [14] [15] Node.js Vs Django: Discover The Top Backend Framework For 2025
https://fullstacktechies.com/node-js-vs-django-backend-framework-2025/

[9] [20] ably.com
https://ably.com/topic/websocket-architecture-best-practices

[10] [13] Free SaaS Startup Tech Stack. The best free tools to launch a fully... | by Joe Osborne | Medium
https://medium.com/@joerosborne/free-saas-startup-tech-stack-e8949047efd3

[11] [17] GitHub - Alpha4Coders/CodeBattle-Arena: Competitive Coding Challenge Platform
https://github.com/Alpha4Coders/CodeBattle-Arena